Sprawozdanie z projektu z „Metod Modelowania Matematycznego”

Wiktor Sowański, Łukasz Sollich
Numery albumów: 203587, 203228

Celem projektu było utworzenie symulatora układu ze sprzężeniem zwrotnym oraz regulatorem PID na pobudzenie trzema rodzajami sygnałów. Jako sygnały w realizacji przyjęto sygnały: harmoniczny, piłokształtny oraz prostokątny.

1 Część Matematyczna

A. Regulator PID

Regulator PID to obiekt posiadający transmitancję:

$$G = K_p \left(1 + \frac{1}{T_i s} + T_d s\right)$$

W zadaniu jednak sprecyzowano, że regulator musi być możliwy do zrealizowania. Jako że idealny człon różniczkujący jest niemożliwy do zrealizowania, transmitancja rzeczywistego regulatora PID przybiera postać:

$$G = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{1 + T_f s}\right)$$

gdzie T_f jest 5-20 razy mniejsza niż T_d. W programie przyjęto wartość $T_f = 0.01$. Transmitancja takiego regulatora wynosi:

$$G_{PID} = K_p + \frac{K_i}{s} + \frac{K_d s}{1 + T_f s} = \frac{(1 + T_f s)K_p s + (1 + T_f s)K_i + K_d s^2}{s(1 + T_f s)} =$$

$$= \frac{s^2(K_d + K_p T_f) + s(K_p + K_i T_f) + K_i}{s^2 T_f + s}$$

B. Obiekt

Transmitancja obiektu wynosi:

$$G = \frac{a_1 s + a_0}{b_2 s^2 + b_1 s + b_0}$$

Z twierdzenia Masona mamy:

$$H = \frac{G_{PID} G_{obiektu}}{1 + G_{PID} G_{obiektu}}$$

Symulacja zaimplementowana została w oparciu o numeryczną metodę Eulera, z krokiem dt = 0.01s, co pozwala na uzyskanie przybliżonej odpowiedzi układu na zadany sygnał. Metoda polega na rysowaniu prostokątów pod krzywą odpowiedzi które przy zmniejszaniu kroku czasowego lepiej opisują kształt odpowiedzi układu. Dokładność symulacji będzie zależała od wyboru kroku czasowego. Mniejszy krok czasowy zwiększa dokładność, ale również wydłuża czas obliczeń.

C. Testy

Symulacja została przetestowana na otwartej pętli regulatora (Parametry regulatora ustawione na 0, człon proporcjonalny ustawiony na 1). Otrzymano wynik dążący do stałej wartości dla biegunów o wartościach leżących w lewej półpłaszczyźnie i dąży do nieskończoności dla biegunów leżących na prawej półpłaszczyźnie. Obiekt zachowuje się więc zgodnie z przewidywaniami.

2 Część Programistyczna

A. Biblioteki

Przy realizacji projektu wykorzystano następujące biblioteki:

- **Tkinter** – do oprawy graficznej
- **Numpy** – do obliczeń matematycznych
- **Matplotlib** – do rysowania wykresów sygnału i odpowiedzi

B. Interfejs Graficzny

Interfejs graficzny został zaprogramowany z wykorzystaniem biblioteki Tkinter pozwalającej na modularne budowanie oprawy graficznej. Po lewej stronie GUI umieszczono miejsce do wprowadzania parametrów symulacji oraz wyboru sygnału zadanego. Do dyspozycji użytkownik ma trzy typy sygnałów: harmoniczny, prostokątny i piłokształtny. Po wybraniu typu sygnału program pokazuje miejsce do wprowadzenia parametrów wybranego sygnału. Po prawej stronie znajduje się wykres odpowiedzi układu oraz sygnału zadanego, dla pokazania wpływu obiektu oraz regulatora.

C. Sygnały

Program w zależności od typu sygnału prosi o odpowiednie dla sygnału parametry (np. Poziom wypełnienia dla prostokąta). W zależności od typu sygnału program generuje sygnał zgodnie z wymaganymi parametrami.

- **Sygnał harmoniczny** generowany jest z wykorzystaniem funckji sinus.
- **Sygnał prostokątny** generowany jest z wykorzystaniem funkcji `np.where()`, gdzie sprawdzany jest warunek czy wypełnienie jest większe od fazy sygnału. Jeśli tak to sygnał przyjmuje wartość amplitudy, a jeśli nie to 0.

- **Sygnał piłokształtny** generowany jest z wykorzystaniem funkcji `np.where()`, gdzie podobnie jak w przypadku sygnału prostokątnego sprawdzany jest warunek czy szerokość jest większa niż faza sygnału. Najpierw jednak następuje sprawdzenie czy nie wsytąpi dzielenie przez zero.

```
if sig_type == "Harmoniczny":
    amp = float(signal_frame.entries["amp"].get())
    omega = float(signal_frame.entries["omega"].get())
    phi = float(signal_frame.entries["phi"].get())
    u = amp * np.sin(omega * t + phi)

 elif sig_type == "Prostokąt":
    amp = float(signal_frame.entries["amp"].get())
    freq = float(signal_frame.entries["freq"].get())
    duty = float(signal_frame.entries["duty"].get())
    phase = (t * freq) % 1.0
    u = amp * np.where(phase < duty, 1.0, 0.0)

elif sig_type == "Piłokształtny":
    amp = float(signal_frame.entries["amp"].get())
    freq = float(signal_frame.entries["freq"].get())
    width = float(signal_frame.entries["width"].get())
    phase = (t * freq) % 1.0

        if width >= 1.0:
            saw_wave = -1.0 + 2.0 * phase
        elif width <= 0.0:
            saw_wave = 1.0 - 2.0 * phase
        else:
            saw_wave = np.where(phase < width, -1.0 + 2.0 * (phase /
               width), 1.0 - 2.0 * ((phase - width) / (1.0 - width)))
u = amp * saw_wave
```

D. Symulacja

Symulacja zaimplementowana została metodą Eulera z krokiem dt = 0.01s. Działa ona w pętli, zapisując wektor odpowiedzi y_{out} zgodnie z wektorem czasu t. Również w każdej iteracji obliczany jest uchyb, zmienna integral będąca sumą prostokątów pod krzywą, oraz zmienna derivative będąca wartością pochodnej błędu. Sygnał sterujący v obliczany jest z wyprowadzonego w części matematycznej wzoru na transmitancję regulatora PID. Obliczane są również pochodne stanów obiektu, wykorzystywane po zcałkowaniu numerycznym w przyszłej iteracji pętli do wyliczenia wyjścia.
Symulacja wykonana jest z wykorzystaniem konstrukcji try-except do obsługi błędów jeśli wystąpią. Uwzględnione zostały błędy formatu danych, struktury układu, oraz inne błędy. Kod symulacji oraz błędów znajduje się poniżej.

```
    y_out = np.zeros_like(t)

            # Stany obiektu
            x1 = 0.0
            x2 = 0.0

            # Zmienne regulatora PID
            integral = 0.0
            derivative = 0.0
            e_prev = 0.0
            v_prev = 0.0

            for i in range(len(t)):
                u_val = u[i]

                # Równanie wyjścia obiektu
                if b2 != 0.0:
                    y_val = a0 * x1 + a1 * x2
                else:
                    # Dla układów I rzędu
                    dx1_temp = (-b0 * x1 + v_prev) / b1 if b1 != 0 else
                        0.0
                    y_val = a0 * x1 + a1 * dx1_temp

                y_out[i] = y_val

                # Uchyb sterowania
                e = u_val - y_val

                # PID - Człon Całkujący
                integral += e * dt

                # PID - Człon Różniczkujący realizowalny
                derivative = (Tf / (Tf + dt)) * derivative + (Kd / (Tf +
                    dt)) * (e - e_prev)

                # Sygnał sterujący PID
                v = Kp * e + Ki * integral + derivative

                # Obliczanie pochodnych stanów obiektu (z równania róż
                    niczkowego)
                if b2 != 0.0:
                    dx1 = x2
                    dx2 = (-b0 * x1 - b1 * x2 + v) / b2

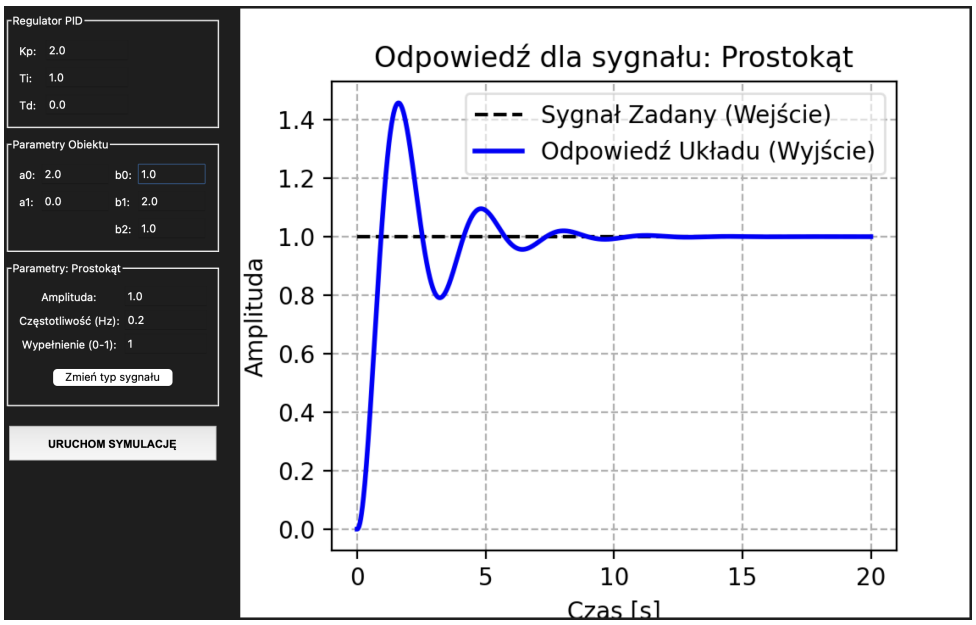
                    # Całkowanie stanów obiektu (Metoda Eulera)
                    x1 += dx1 * dt
                    x2 += dx2 * dt
                else:
                    # Obsługa, gdy układ jest I rzędu (b2 = 0)
                    if b1 != 0.0:
                        dx1 = (-b0 * x1 + v) / b1
                        x1 += dx1 * dt

                # Zapisanie poprzednich wartości
                e_prev = e
                v_prev = v
```

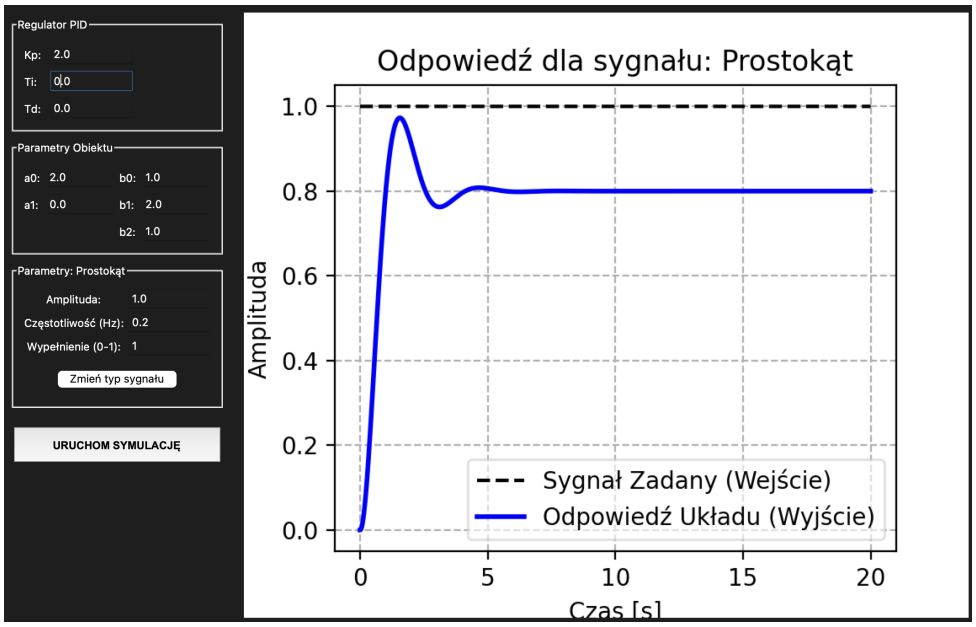
```
except ValueError:
    messagebox.showerror("Błąd danych", "Upewnij się, że wszystkie pola zawierają poprawne liczby (kropki nie przecinki).")
except ZeroDivisionError:
    messagebox.showerror("Błąd struktury","Dzielenie przez zero! Sprawdź parametry układu (np. b1 i b2 równe 0).")
except Exception as e:
    messagebox.showerror("Błąd symulacji", f"Wystąpił nieoczekiwany błąd:\n{str(e)}")
```

E. Rysowanie wykresu

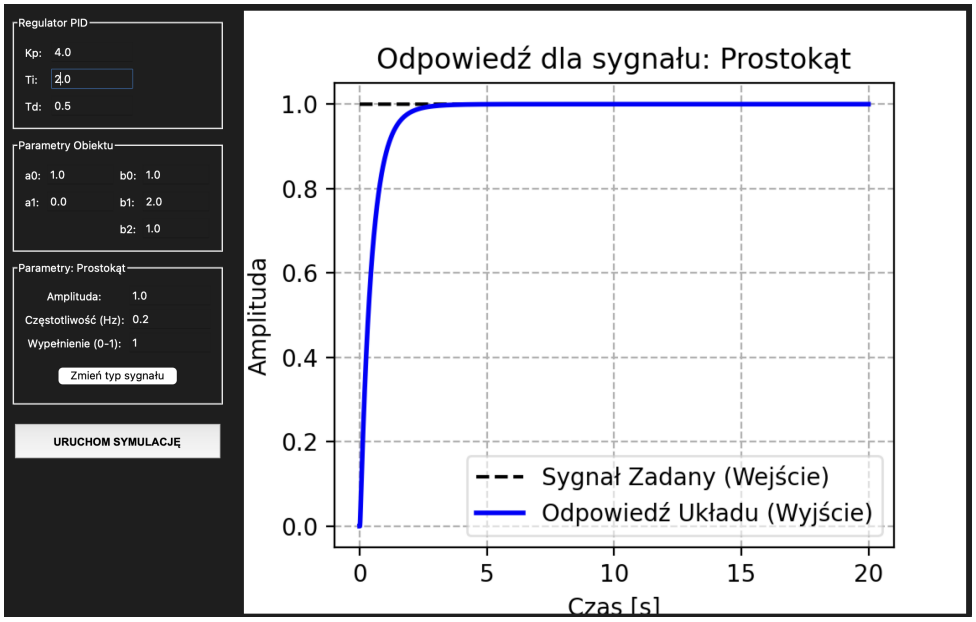
Z wykorzystaniem funkcji z biblioteki Matplotlib narysowano wykresy sygnału zadanego dla wybranych parametrów oraz odpowiedzi układu. Ponizej przedstawiono przykładowe odpowiedzi układów na pobudzenie skokiem.



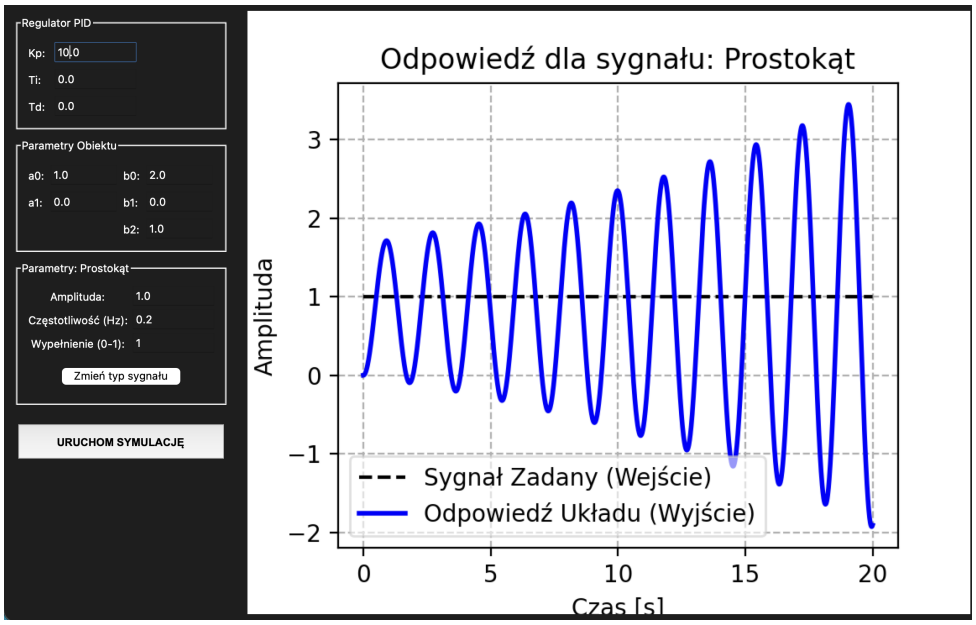
Rysunek 1: Odpowiedź układu z przeregulowaniem i oscylacjami przed osiągnięciem wartości ustalonej. Z astatyzmem rzędu 2.



Rysunek 2: Odpowiedź układu z przeregulowaniem i oscylacjami przed osiągnięciem wartości ustalonej. Śledzący uchyb sterowania, z powodu zbyt niskiego rzędu astatyzmu.



Rysunek 3: Odpowiedź układu z optymalnie dobranymi nastawami regulatora PID.



Rysunek 4: Odpowiedź układu z biegunami na prawej półpłaszczyźnie co powoduje generowanie drgań.

3 Wnioski

Układ generuje odpowiedź na skok jednostkowy zgodnie z przewidywaniami. Wykresy odpowiedzi pokrywają się ze znanymi z zajęć automatyki kształtami, dla różnych pobudzeń można zaobserwować przeregulowanie lub przy wybraniu biegunów na prawej półpłaszczyźnie można zaobserwować niestabilność układu co również zgadza się z teorią. Sprawdzenie poprawności modelu inną metodą jest niemożliwe z powodu braku dostępności mechanicznych bądź elektrycznych układów do symulowania oraz aparatury do mierzenia odpowiedzi rzeczywistych układów.

4 Źródła

1. „Podstawy Automatyki” tom 1. J. Nowakowski; Politechnika Gdańska 1992
2. Podręcznik biblioteki „Scipy”
3. Dokumentacja ze strony „python.org”